PORTFOLIO

Game Client Programmer

강병준

010-2653-1292 / dzzzing@naver.com

Works

01. Team Project Portfolio

듀엣 나이트 어비스

팀장 / Framework / Shader / Boss

https://www.youtube.com/watch?v=EJ9jz6HGmYg

https://github.com/Byungcoco/FinalProject

C++ DirectX11 3D HLSL

PhysX Nlohmann json FMOD

멀릿 매드 잭

팀장 / Map Tool / Boss / Effect

https://www.youtube.com/watch?v=7JCpLUCb3-A

https://github.com/Jusin1/MadMulletJack

C++ DirectX9 3D

Nlohmann json FMOD

02. Personal Project Portfolio

나루토 시노비 스트라이커

https://www.youtube.com/watch?v=Z5cCLy7Tuso

https://github.com/Byungcoco/D3D11_Practice

C++ DirectX11 3D HLSL

Nlohmann json FMOD

카타나 제로

https://www.youtube.com/watch?v=IY5oGaIb_kI

https://github.com/Byungcoco/KatanaZero_Copy

C++ WinAPI 2D

03. Work Experience

KTds VirtualOffice

UI / Bug fix / Feature Development

https://www.youtube.com/watch?v=cD_pEctRQ3w

C# Unity Metaverse Solution

Map Editor Addressable

4Fun Speaking

Content Development / Planning

https://app.wadiz.io/links/q37xZkzB8D

Lua Roblox Studio 3D

Language Teaching

Index

01. AI 활용

a. 로딩 속도 개선 —— 04p ~ 06p

02. System

a. 델리게이트 · 이벤트 시스템 —— 07p ~ 14p
b. Loose Octree —— 15p ~ 17p
c. 툴 파싱 파이프라인 —— 18p ~ 19p
d. 다각형 Navigation Cell —— 20p ~ 22p
e. 언리얼 맵 데이터 좌표계 변환 —— 23p ~ 24p
f. 애니메이션 블렌딩 —— 25p ~ 26p

03. Contents

-게임 플레이

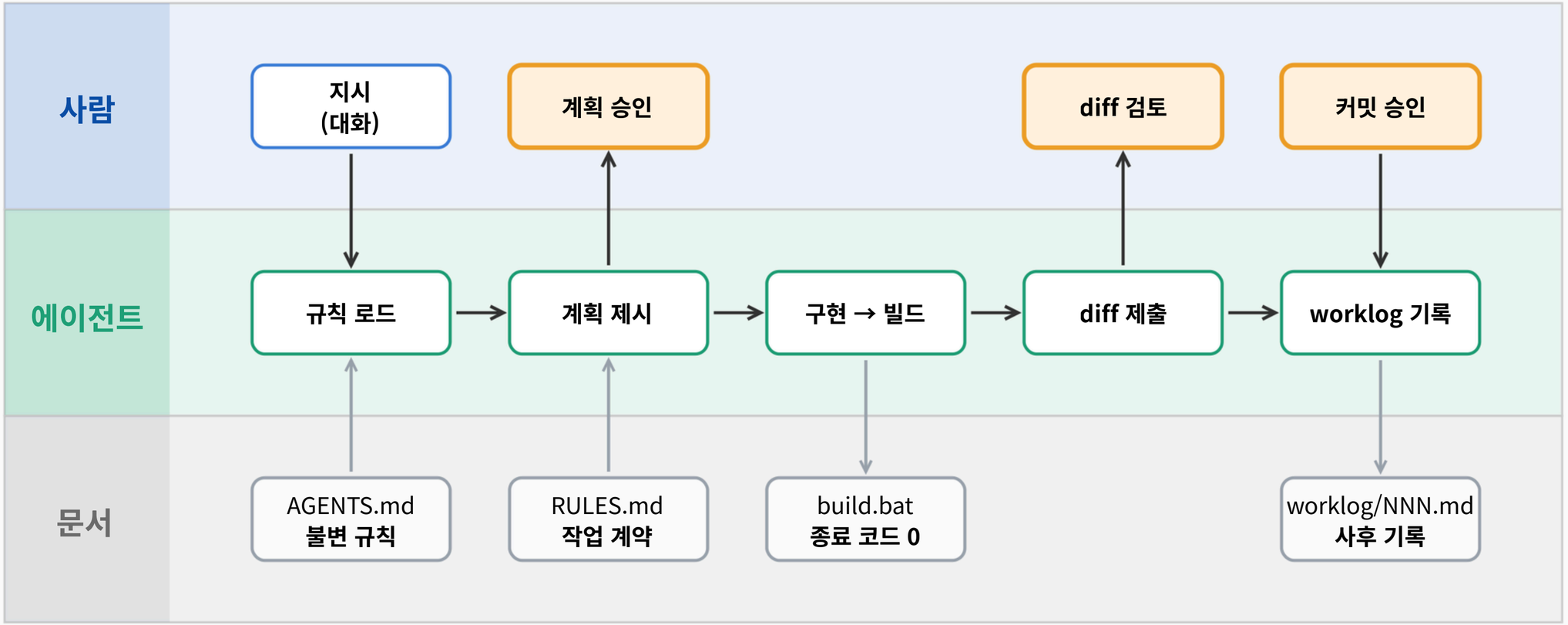
a. 리플레이 —— 27p ~ 28p
b. Prefab을 활용한 무한맵 —— 29p ~ 30p
d. 캐릭터 커스텀 —— 31p ~ 32p
e. 벽타기 —— 33p ~ 36p

04. 그 외

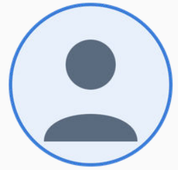
01. AI 활용 로딩속도 개선

어떻게?

- 자체 엔진의 로딩 구조 개선을 **AI 코딩 에이전트(Codex, Claude Code)와의 협업**으로 수행
- 에이전트에게 작업을 맡기기 전에 **안전하게 프로젝트를 수정하도록 규칙 체계를 먼저 설계**



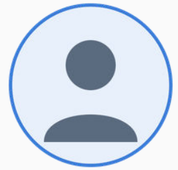
01. AI 활용 로딩속도 개선



AGENTS.md와 RULES.md를 먼저 읽고 시작해. 목표는 로딩속도 개선이고 이번 001은 기준선 계측이야
로딩 동작 변경 금지, 측정 코드 추가만 허용, 실행을 통한 검증은 내가 직접한다.



숙지했습니다.
① Engine_Utils에 steady_clock RAII 타이머
② 모델·텍스처·JSON 파일별 측정
③ loading_profile.txt 출력
. . .
예상 개선 시간은 약 20초입니다.



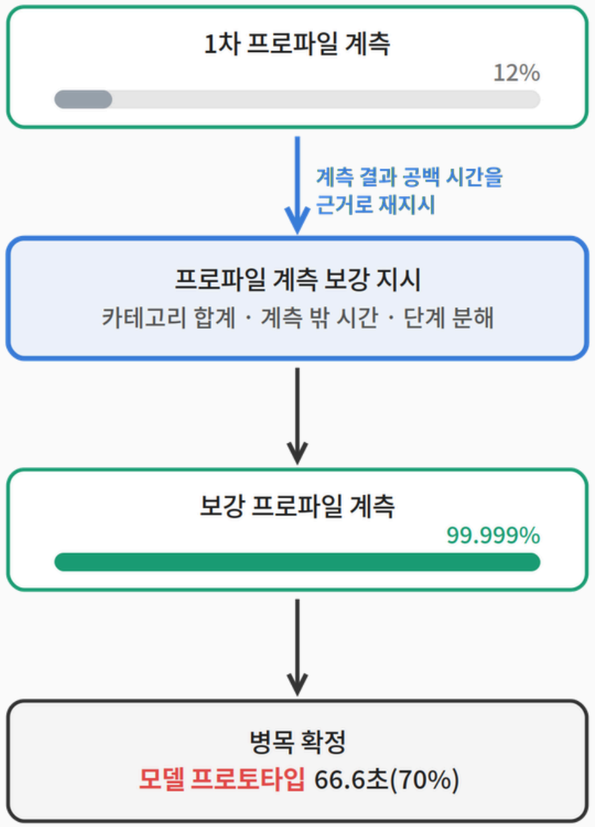
1. Profile.txt 확인 결과 모델간 병렬화가 아직 이루어지지 않았고, 파악했던 모델 개수에 대한 오류로 예상 개선시간에서 크게 벗어나.
2. Level 별로 파악할 수 있게 파일명으로 구분해서 저장해
3. CModel 클래스의 Create 경로에 공유 상태 접근이 있으면 구현하지 말고 보고해

. . .



네, 알겟습니다. ResourceManager의 map은 mutex가 없어 동시 insert시 트리가 깨질 위험이 있습니다.
캐시 안전화 선행을 권장하며 5개의 파일 제한을 넘기 때문에 3단계 게획 분할로 진행하겠습니다.

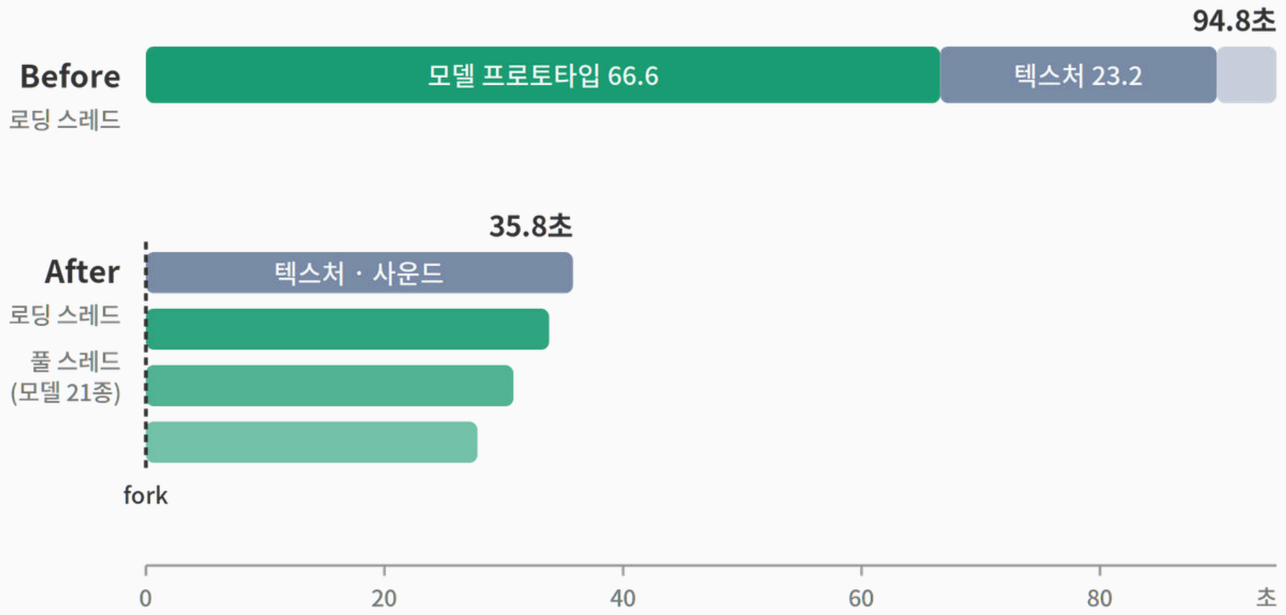
01. AI 활용 로딩속도 개선



▲에이전트의 첫 결과물을 그대로 받지 않고, 데이터의 공백을 근거로 프로파일 계측을 두차례 보강 지시

→ 병목의 실체를 확실하게 추적

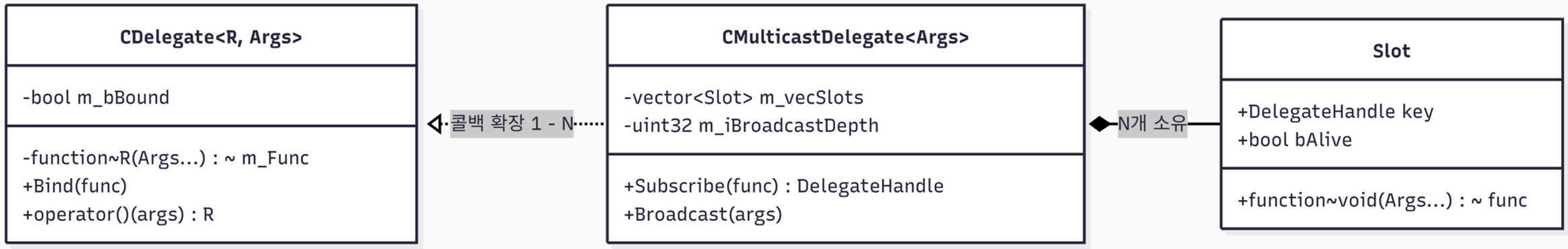
결과



로고 레빌 로딩 총 시간 **94.8초 → 35.8초**

2.6배 단축

02. System 델리게이트·이벤트 시스템



▲ 단일 콜백(CDelegate)을 Slot 배열로 확장한 1:N 멀티캐스트 구조

왜?

- 기존 이벤트 처리 방식이 객체간 결합도가 높은 문제 발생
- 문자열로 관리하다가 오타·잘못된 인자로 런타임 크래시 발생

설명

- CDelegate : std::function을 감싼 1:1 콜백
- CMulticastDelegate::Slot을 vector로 소유해 여러 구독자에게 Broadcast (1 : N)
- 가변 템플릿(R, Args...)로 콜백 시그니처를 타입에 고정

장점

- 문자열 키 → 타입 기반으로 전환해 오타·타입 불일치를 컴파일 타임에 잡아냄
- 호출자와 수신자가 서로를 몰라도 연결하여 결합도 감소

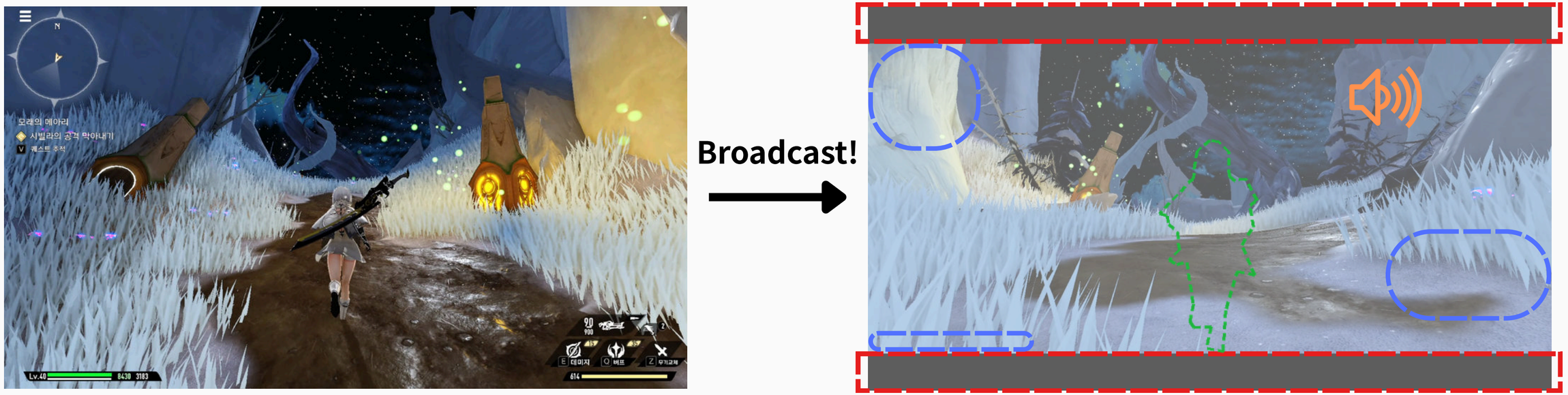
```
template<typename R, typename... Args>
class CDelegate<R(Args...)>
{
        ...중략
        void Bind(std::function<R(Args...)> func)
        {
                m_Func = std::move(func);
        }
        R operator()(Args... args) const
        {
                return m_Func(std::forward<Args>(args)...);
        }
}
```

▲ CDelegate 클래스 일부

02. System 델리게이트·이벤트 시스템

활용 1. EventBus Manager

- MulticastDelegate를 활용해 전역 이벤트를 관리하는 매니저
- typeid(Tag)로 태그별 채널을 중복 없이 관리, template + using으로 시그니처를 타입 안전하게 라우팅

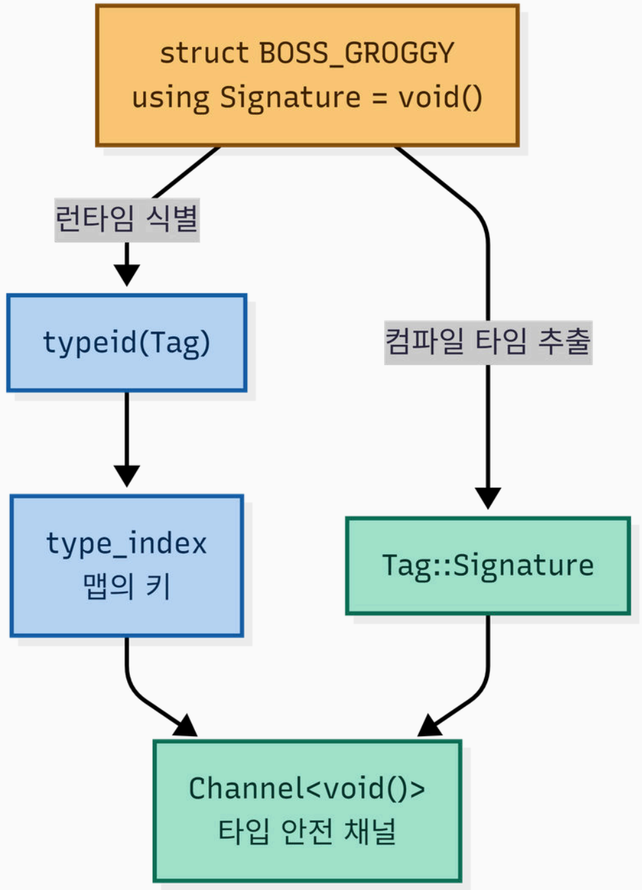


▲ **EventBus 활용 예시:** 메인카메라를 시네마틱 카메라로 변경 연출한 인게임 스샷

LetterBox 연출, **UI Invisible**, **캐릭터 입력 비활성화**, **BGM 변경**

02. System 델리게이트·이벤트 시스템

활용 1. EventBus Manager



A. 이벤트 정의: 타입, using으로 시그니처를 각인

```
struct BOSS_GROGGY { using Signature = void(); };
```

B. 구독: 멤버 함수 바인딩

```
DelegateHandle handle =
m_pGameInstance->Subscribe<BOSS_GROGGY>(pUI, &CBossUI::OnGroggy);
```

C. 발행: typeid로 채널을 찾아 Broadcast

```
m_pGameInstance->Broadcast<BOSS_GROGGY>();
```

D. 구독 해제: 구독시 반환 받은 handle로 구독 해제

```
m_pGameInstance->Unsubscribe<BOSS_GROGGY>(handle);
```

▲ CEventBus_Manager의 핵심 함수 사용 예시

02. System 델리게이트·이벤트 시스템

```
template<typename Tag, typename T, typename... Args>
DelegateHandle Add_MemFunc(T* pObj, void(T::*memFunc)(Args...) const)
{
        using Sig = typename Tag::Signature;
        static_assert(std::is_same_v<Sig, void(Args...)>, "signature not match");
        return Subscribe<Tag>([pObj, memFunc](Args... args)
                {
                        (pObj->*memFunc)(std::forward<Args>(args)...);
                });
}
```

Tag의 콜백 시그니처를 컴파일 타임에 추출

등록하려는 함수 시그니처가 태그와 다른지 **컴파일 타임에 검사**

▲ CEventBus_Manager의 Add_MemFunc

```
template<typename Tag>
DelegateHandle Subscribe(std::function<typename Tag::Signature> func)
{
        auto* channel = GetOrCreate<Tag>();
        return channel->multicast.Subscribe(std::move(func));
}

template<typename Tag>
Channel<typename Tag::Signature>* GetOrCreate()
{
        const std::type_index key{ typeid(Tag) };
        auto itr = m_channels.find(key);
          ...중략
}
```

Tag로 찾은 채널의 멀티캐스트에 콜백 등록
move로 함수 객체를 복사 없이 전달

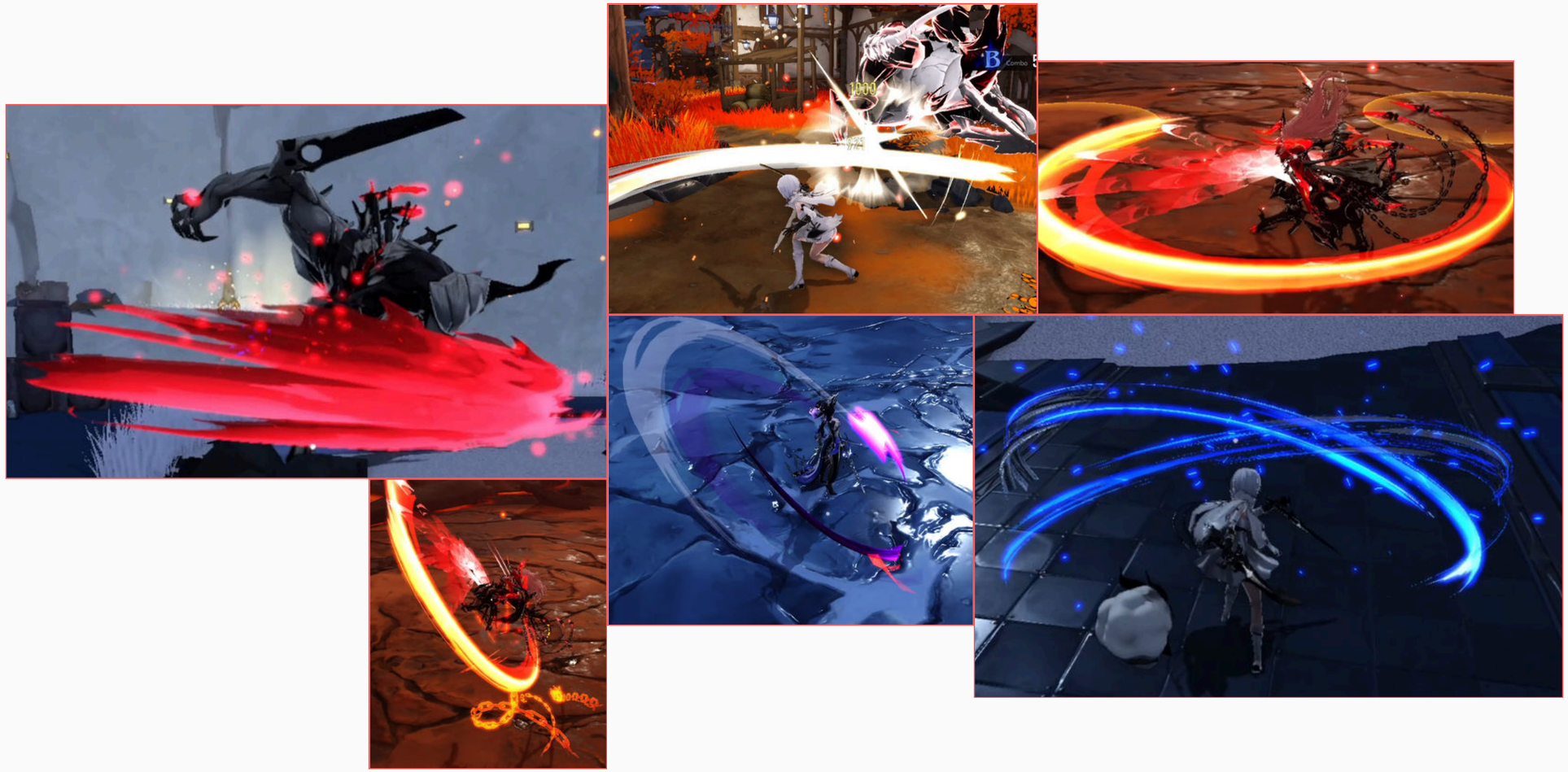
Tag 타입 자체를 런타임 키로 변환
이벤트마다 고유한 type_index가 나와 채널을 식별

type_index를 키로 std::map에서 채널 조회
→ **O(logN) 탐색을 의도**, 태그별 채널을 중복 없이 관리

▲ CEventBus_Manager의 Subscribe와 GetOrCreate

02. System 델리게이트·이벤트 시스템

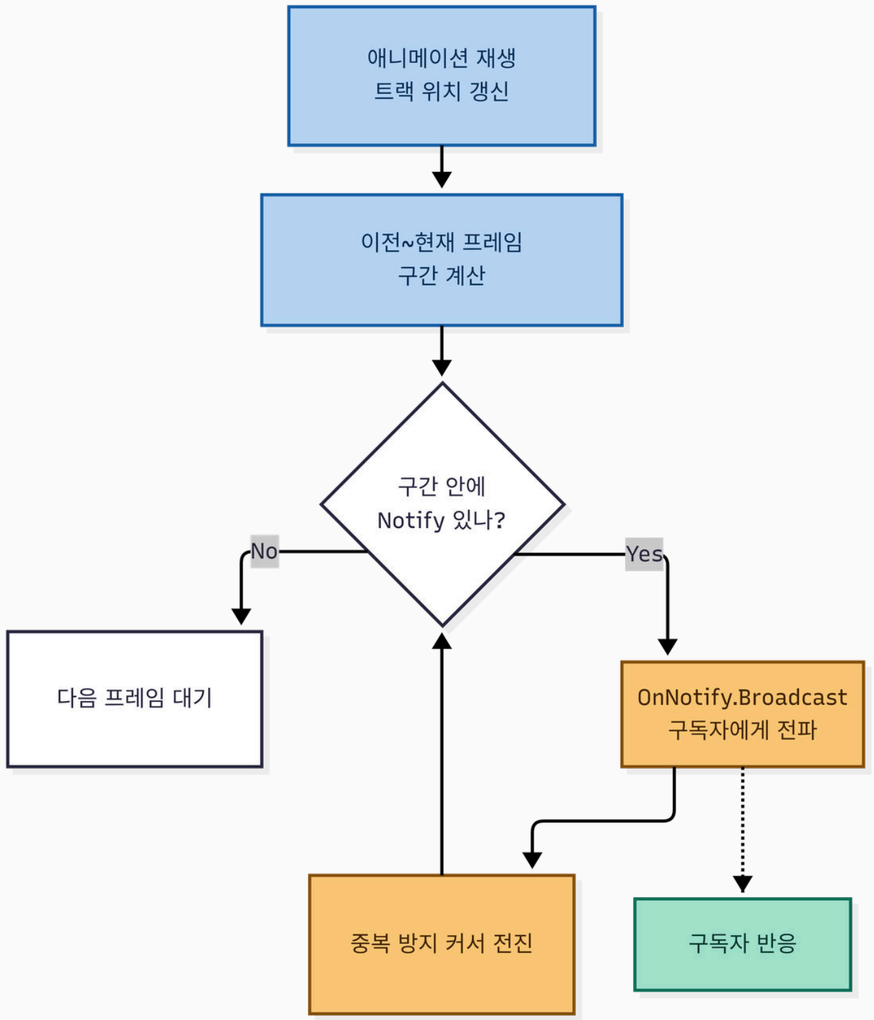
활용 2. - Animation Notify



▲ Animation Notify 활용 인게임 스크린샷

02. System 델리게이트·이벤트 시스템

활용 2. - Animation Notify




```
enum class EAnimNotifyPhase
{
    Immediatley = 0,
    Late,
    PreRender,
    END
};
```

```
vector<AnimNotifyKey> m_vecNotifies[ENUM_TO_UINT(EAnimNotifyPhase::END)];
```

→ 즉시, Late Update, 렌더 직전으로 3개의 Phase로 분리하여 이중배열에 NotifyKey를 넣어 관리

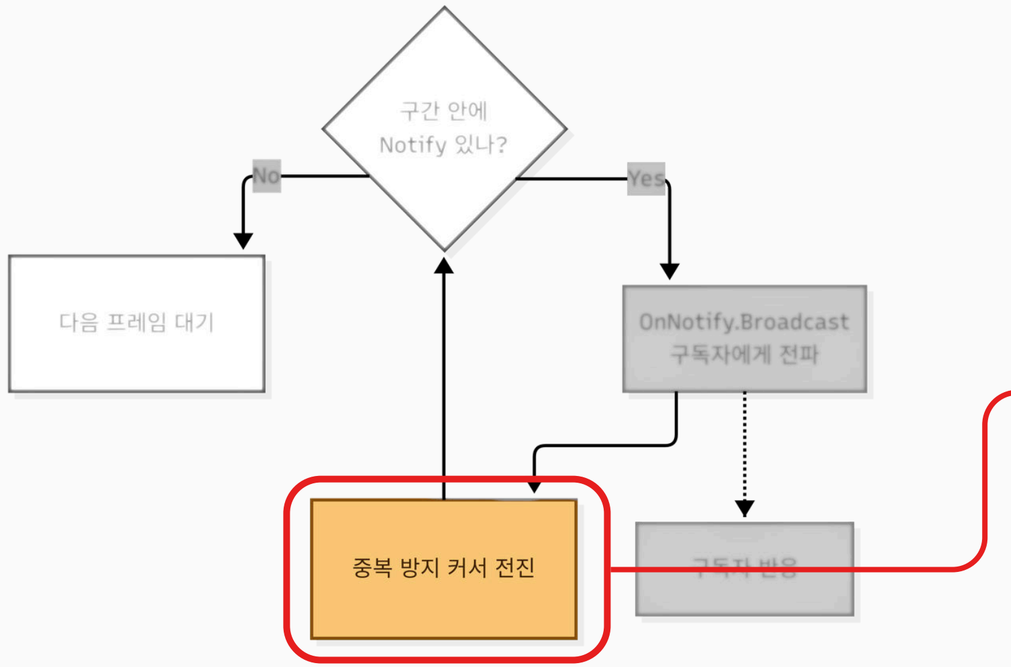
```
struct AnimNotifyKey
{
    EAnimNotifyId eID{ EAnimNotifyId::Effect };
    float fTrackPosition{ 0.f };
    unsigned int  iParam0{ 0 };
    unsigned int  iParam1{ 0 };
    float         fParam0{ 0.0f };
    float         fParam1{ 0.0f };
    bool          bParam0{ false };
    bool          bParam1{ false };
};
```

Notify 종류마다 다른 값을 하나의 구조체로 담기위해 공용 파라미터를 사용

→ 하나의 구조체로 여러 종류의 Notify를 표현

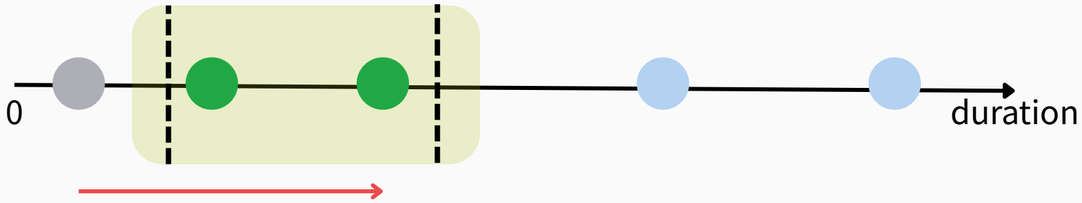
02. System 델리게이트·이벤트 시스템

활용 2. - Animation Notify



- Notify **완료**
- Notify **중**
- Notify **대기**

From TrackPosition. **To** TrackPosition.



현재 프레임 기준 Cursor 전진

왜?

- 완료된 Notify가 다시 Broadcast 되는 문제가 프레임마다 반복 발생

장점

- Notify가 완료된 구간까지의 인덱스로 Cursor로 저장하여 다음 프레임은 Cursor 이후부터 검사하여 중복 방지
- 매번 처음부터 검사하지 않아 다음 Notify만 확인
  → (배열의 **임의접근으로 O(1)**)

02. System 델리게이트·이벤트 시스템

```
void CModel::Emit_Notifies(CModelAnimation* pAnimation, float fCurPos, EAnimNotifyPhase ePhase)
{
    const auto& notifyKeys = pAnimation->Get_Notifies(ePhase);
    if (notifyKeys.empty()) return;

    const float fDuration = pAnimation->Get_DurationTime();

    uint iIndex = pAnimation->Get_NotifyCursor(ePhase);

    auto Emit_Range = [&](float fFrom, float fTo)->void
    {
        while (iIndex < notifyKeys.size() &&
               notifyKeys[iIndex].fTrackPosition <= fTo)
        {
            if (notifyKeys[iIndex].fTrackPosition > fFrom)
                OnNotify.Broadcast(notifyKeys[iIndex]);

            pAnimation->Set_NotifyCursor(ePhase, ++iIndex);
        }
    };

    if (!m_bLooped)
        Emit_Range(m_fAnimPrevTrackPosition, fCurPos);
    else
    {
        Emit_Range(m_fAnimPrevTrackPosition, fDuration);
        pAnimation->Set_NotifyCursor(ePhase, 0);
        iIndex = 0;
        Emit_Range(0.f, fCurPos);
    }
}
```

현재 Phase의 최선 Cursor 위치 Index

From ~ To 구간에 걸친 Notify만 Broadcast하는 람다

커서 인덱스 전진

루프 애니메이션의 끝 ~ 시작 경계를 두번에 나눠서 처리

02. System Loose Octree

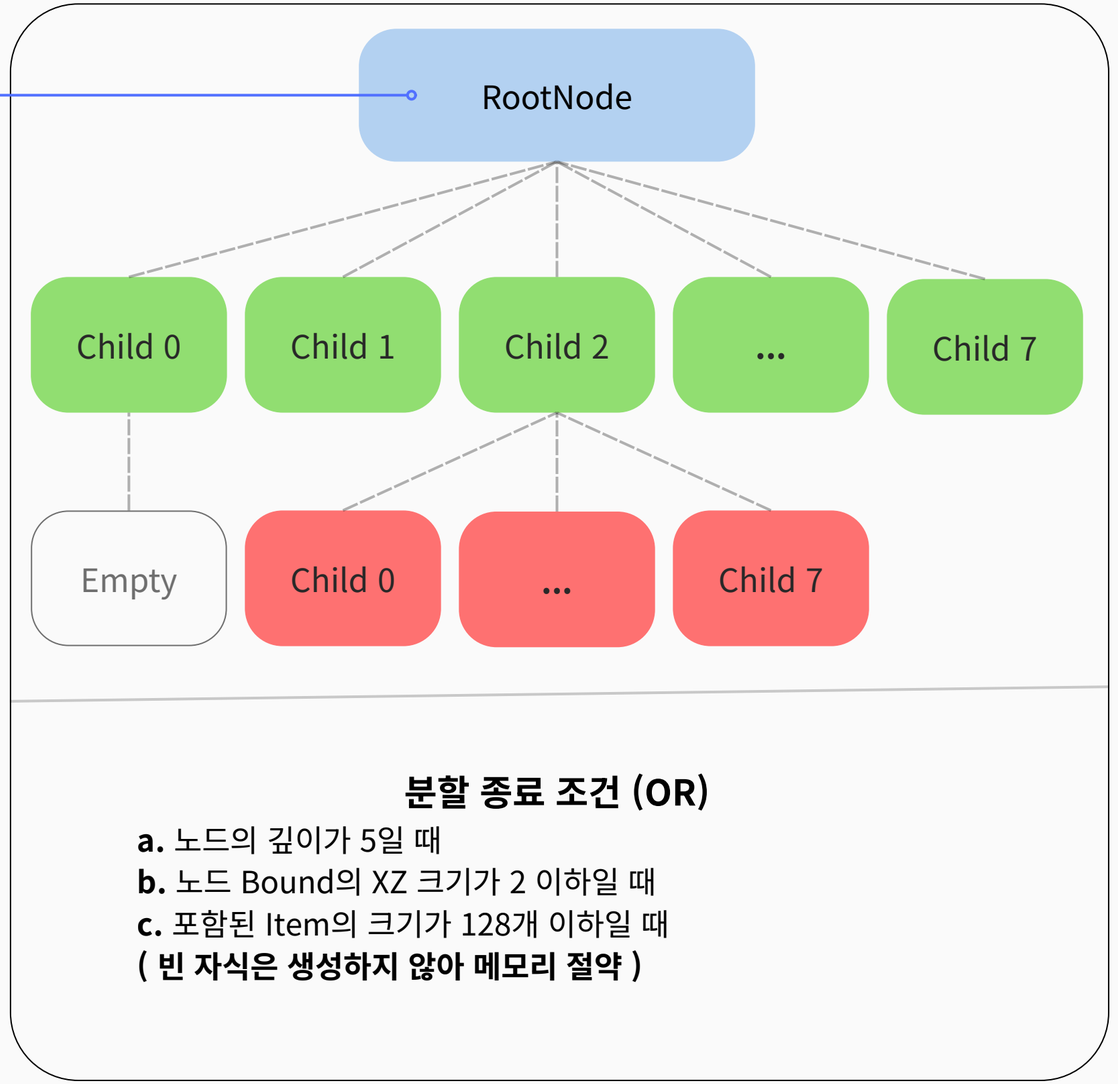
왜?

- 정적 오브젝트가 많은 넓은 맵의 공간을 분할해 화면 밖 영역을 통째로 걸러내고자 함
- 노드간 경계에 걸친 객체가 상위 노드로 몰리는 문제가 발생해, 경계를 **1.3배로 느슨(loose)**하게 잡아 이를 해결함

▼ Octree의 node 구조체

```
#define DIVISIOIN 8
typedef struct tagOctreeNode
{
	BoundingBox bounds;
	BoundingBox looseBounds;
	_int iDepth{ 0 };
	tagOctreeNode* pChilds[DIVISION]{nullptr};
	vector<tagOctreeEntry*> vecItems;

	_bool Is_Leaf() const
	{
		for (_int i = 0; i < DIVISION; ++i)
			if (pChilds[i] != nullptr)
				return false;
		return true;
	}
}OCTREE_NODE;
```



분할 종료 조건 (OR)

a. 노드의 깊이가 5일 때
b. 노드 Bound의 XZ 크기가 2 이하일 때
c. 포함된 Item의 크기가 128개 이하일 때
(빈 자식은 생성하지 않아 메모리 절약)

02. System Loose Octree

```
void COctree_Manager::Insert(OCTREE_NODE* pNode, OCTREE_ENTRY* pEntry)
{
    if (pNode->looseBounds.Contains(pEntry->AABB) == ContainmentType::DISJOINT) {
        Add_ToNode(pNode, pEntry);
        return;
    }

    const bool bCanSplit = pNode->iDepth < m_Desc.iMaxDepth;

    if (pNode->Is_Leaf() == true) {
        if (!bCanSplit || pNode->vecItems.size() < m_Desc.iMaxItemsPerLeaf) {
            Add_ToNode(pNode, pEntry);
            return;
        }
        bool bTooSmall = (pNode->bounds.Extents.x <= m_Desc.fMinNodeSizeXZ) ||
                         (pNode->bounds.Extents.z <= m_Desc.fMinNodeSizeXZ);
        if (bTooSmall) {
            Add_ToNode(pNode, pEntry); return;
        }
        else
            Split(pNode);
    }

    const _int iIndex = Candidate_ChildIndex(pNode->bounds, pEntry->AABB);
    if (pNode->pChilds[iIndex] == nullptr) {
        BoundingBox childrenBox = Make_ChildBounds(pNode->bounds, iIndex);
        pNode->pChilds[iIndex] = Create_Node(childrenBox, m_Desc.fLooseFactor, pNode->iDepth + 1);
    }

    if (pNode->pChilds[iIndex]->looseBounds.Contains(pEntry->AABB) == ContainmentType::CONTAINS)
        Insert(pNode->pChilds[iIndex], pEntry);
    else
        Add_ToNode(pNode, pEntry);
}
```

looseBound의 경계 밖이면 현재 노드가 보관

종료조건 a. 노드의 깊이가 5일 때

종료조건 b. 아이템이 128개 미만이면 분할없이 보관 **(지연 분할)**

종료조건 c. 노드 XZ 크기가 작으면 더 이상 분할하지 않음

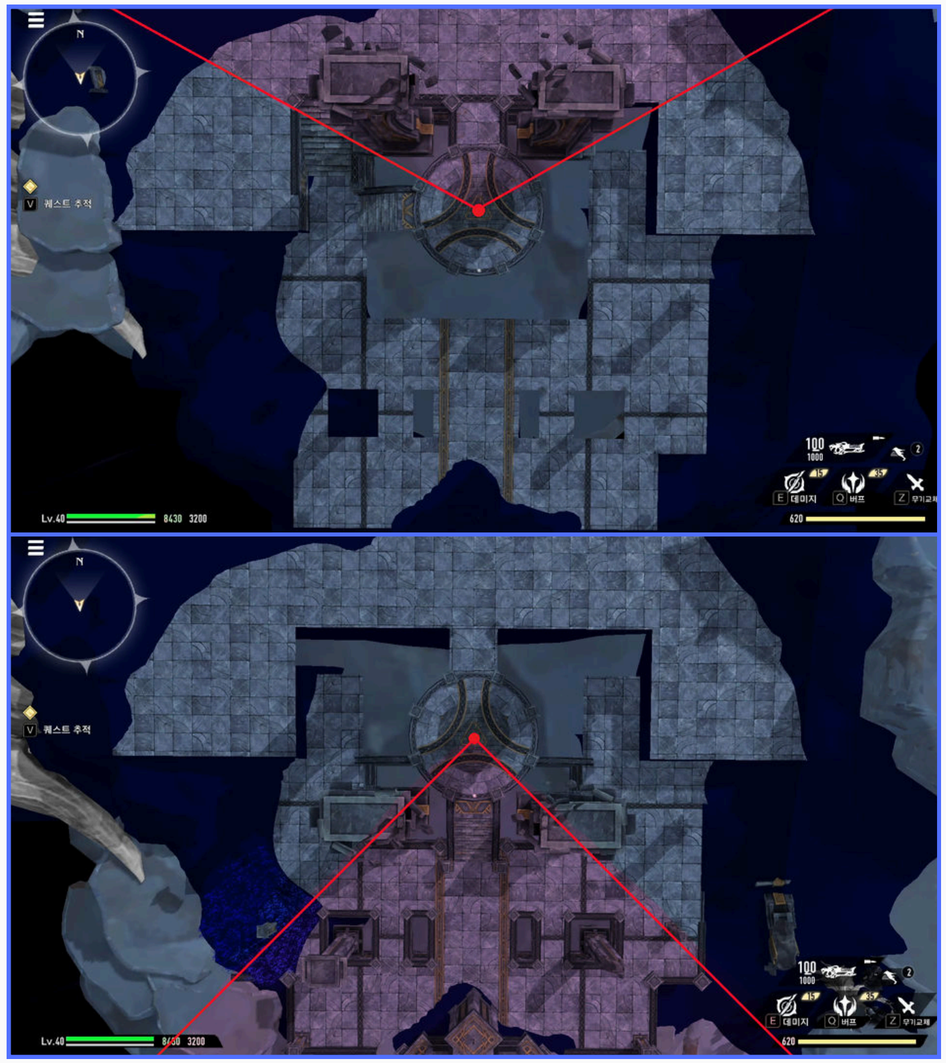
```
_int COctree_Manager::Candidate_ChildIndex(const BoundingBox& node,
const BoundingBox& object)
{
    _int iX = (object.Center.x >= node.Center.x) ? 1 : 0;
    _int iY = (object.Center.y >= node.Center.y) ? 1 : 0;
    _int iZ = (object.Center.z >= node.Center.z) ? 1 : 0;

    return (iX) | (iZ << 1) | (iY << 2);
}
```

*세축 비교를 비트로 조합해 규칙성 있는 Index 생성

자식 looseBound의 경계가 완전히 품으면 재귀 삽입
경계에 걸치면 현재 노드가 보관

02. System Loose Octree



▲ 탑다운 뷰로 확인한 절두체 컬링 스크린샷

```
void COctree_Manager::Query_Node(OCTREE_NODE* pNode, const BoundingFrustum& frustrum,
RENDER_CATEGORY eCtegory, OUT vector<CGameObject*>& outObjects) const
{
        if (pNode == nullptr)
                return;

A.      if (frustrum.Contains(pNode->looseBounds) == ContainmentType::DISJOINT)
                return;

B.      for (auto* pEntry : pNode->vecItems)
        {
                if (eCtegory != pEntry->eCategory)
                        continue;

                if (frustrum.Contains(pEntry->AABB) != ContainmentType::DISJOINT)
                        outObjects.push_back(pEntry->pGo);
        }

C.      for (_int i = 0; i < DIVISION; ++i)
        {
                if (pNode->pChilds[i] != nullptr)
                        Query_Node(pNode->pChilds[i], frustrum, eCtegory);
        }
}
```

A. 노드의 looseBounds 경계가 Frustrum 밖이면 이 **서브 트리 전체를 스킵**
B. 노드 안 아이템만 **개별 Frustrum 판정**
C. 살아남은 노드만 **자식으로 재귀**

02. System 툴 파싱 파이프라인

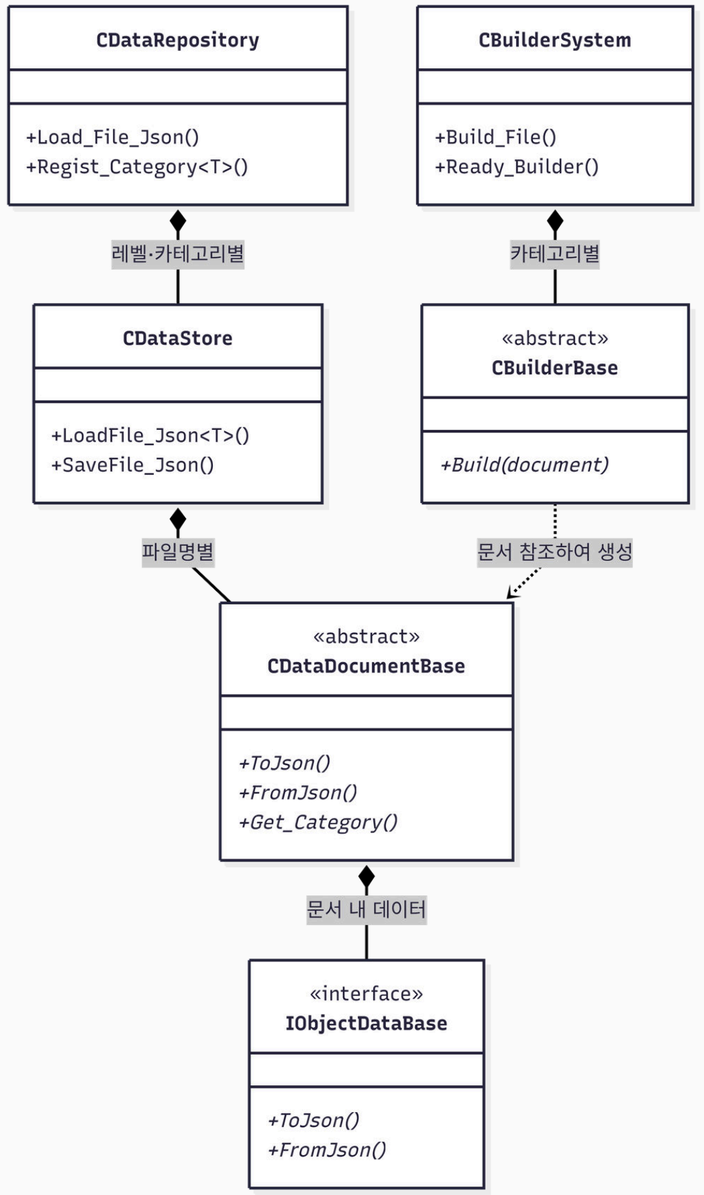
왜?

- 팀원마다 개별 툴 (맵·이펙트·UI·애니메이션 등)을 만들어 데이터 형식이 제각각
- 툴 카테고리마다 로딩 방식이 달라 관리가 어렵고, 새 툴이 생길 때마다 코드를 새로 작성하는 문제 발생

장점

- 어떤 툴 데이터든 **Regist → Load → Build 3단계로 통일**
- 데이터 저장(Repository)과 게임 객체 생성(Builder)을 분리
- 타입만 등록하면 동작하는 구조로 팀원들의 서로 다른 툴 데이터를 하나의 규칙으로 통합

```
HRESULT CLevel_Test::Ready_Map()
{
        중략...
        if (FAILED(m_pGameInstance->Regist_Document<CDataDocument_Map>(iLevelID, eCategory)))
                return E_FAIL;
        if (FAILED(m_pGameInstance->Load_File_Json(iLevelID, eCategory, FilePath)))
                return E_FAIL;
        if (FAILED(Build_File(iLevelID, eCategory, FilePath.stem())))
                return E_FAIL;
        return S_OK;
}
```



02. System 툴 파싱 파이프라인

```cpp
template<typename T>
HRESULT CDataRepository::Regist_Category(_uint iLevelID, DTO::ECategory eCategory)
{
	static_assert(std::is_base_of_v<CDataDocumentBase, T>,
		"T is not derived from CDataDocumentBase");

	CDataStore* pStore = CDataStore::Create();

	TStoreEntry entry;
	entry.pStore = pStore;

	entry.load = [](CDataStore* pStore, const path& filePath) -> HRESULT {
		return pStore->LoadFile_Json<T>(filePath);
	};
	entry.create = []() -> CDataDocumentBase* {
		return T::Create();
	};

	m_mapStores[iLevelID][eCategory] = std::move(entry);
	return S_OK;
}
```

```cpp
using DataRepository_LoadFunc
	= std::function<HRESULT(CDataStore*, const path&)>;
using DataRepository_CreateFunc
	= std::function<CDataDocumentBase*()>;

struct StoreEntry
{
	CDataStore* pStore{nullptr};
	DataRepository_LoadFunc load;
	DataRepository_CreateFunc create;
};
```

***타입별 로드·생성을 std::function으로 묶은 카테고리 단위 구조체**
→ 타입별 로드·생성 방법을 람다로 저장

레벨별·카테고리별로 함수 묶음을 저장

```cpp
Regist_Category<CDataDocument_Effect>(levelID, ECategory::EFFECT);

Load_File_Json(levelID, ECategory::EFFECT, filePath);
```

등록시에만 타입을 명시

이후 로딩은 카테고리만! 내부에서 바인딩 된 람다가 실행

▲ Regist_Category의 호출부

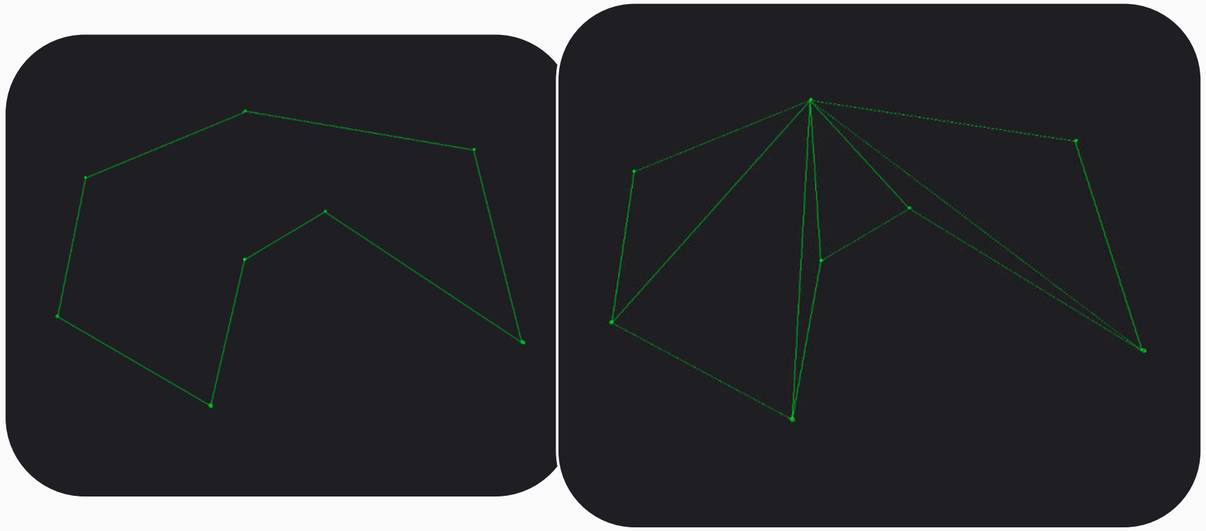
02. System 다각형 Navigation Cell

왜?

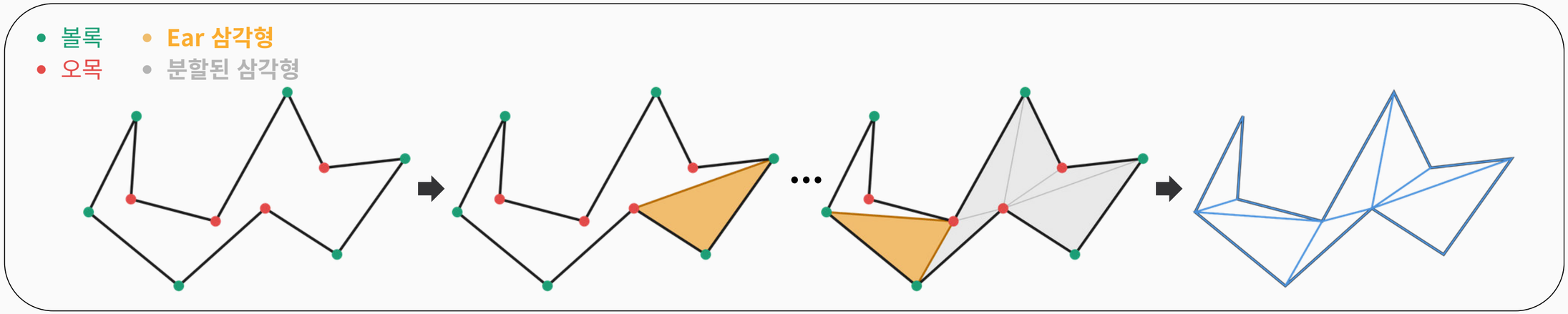
- 맵에 Navigation Cell을 삼각형 단위로 설치하는 반복적인 작업을 줄이기 위해 **Ear Clipping 알고리즘**을 사용

Ear Clipping?

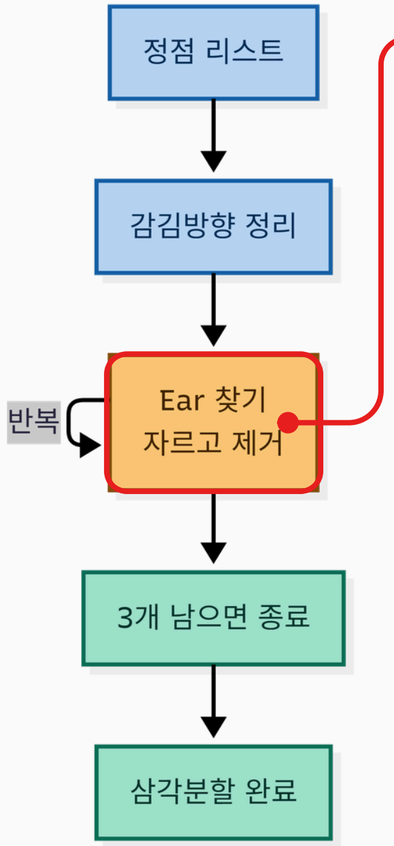
- 연속된 세 정점이 만드는 삼각형 중, 볼록하고 내부에 다른정점이 없어 잘라낼 수 있는 삼각형
  → N개의 정점으로 N-2개 삼각형으로 분할



▲ 툴에서의 예시 결과물 스크린샷



02. System 다각형 Navigation Cell




```
_bool Is_Ear(const vector<_float3>& vecPoints,
             const vector<_int>& vecIndexList, _int i, _bool bCCW)
{
    const _int iCount = static_cast<_int>(vecIndexList.size());
    const _int iIndex_0 = vecIndexList[(i + iCount - 1) % iCount];
    const _int iIndex_1 = vecIndexList[i];
    const _int iIndex_2 = vecIndexList[(i + 1) % iCount];

    _float3 vA = vecPoints[iIndex_0];
    _float3 vB = vecPoints[iIndex_1];
    _float3 vC = vecPoints[iIndex_2];

    _float fCross = Compute_Cross2DXZ(vA, vB, vC);
    _float fOrientSign = bCCW ? 1.f : -1.f;
    if (fCross * fOrientSign <= 0.f)
        return false;

    for (_int j = 0; j < iCount; ++j)
    {
        const _int iK = vecIndexList[j];
        if (iK == iIndex_0 || iK == iIndex_1 || iK == iIndex_2)
            continue;
        if (Point_InTriangleXZ(vecPoints[iK], vA, vB, vC))
            return false;
    }
    return true;
}
```

```
_float Compute_Cross2DXZ(const _float3& vA, const _float3& vB, const _float3& vC)
{
    const _float AB_X = vB.x - vA.x;
    const _float AB_Z = vB.z - vA.z;
    const _float BC_X = vC.x - vB.x;
    const _float BC_Z = vC.z - vB.z;

    return AB_X * BC_Z - AB_Z * BC_X;
}
```

*볼록(Convex) 판정을 하는 함수
벡터별 XZ 평면에서의 외적 부호가 감김방향과 같은지 판별

```
_bool Point_InTriangleXZ(const _float3& vP,
                         const _float3& vA, const _float3& vB, const _float3& vC)
{
    auto sign = [](auto& p1, auto& p2, auto& p3) {
        return (p1.x - p3.x) * (p2.z - p3.z) - (p2.x - p3.x) * (p1.z - p3.z);
    };

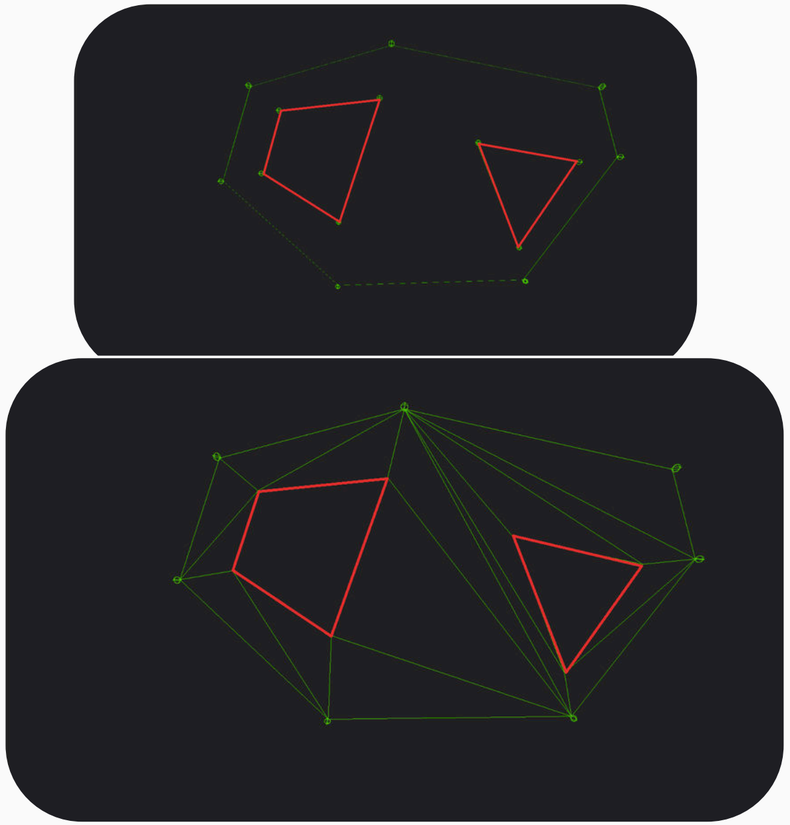
    _float d1 = sign(vP, vA, vB), d2 = sign(vP, vB, vC), d3 = sign(vP, vC, vA);
    ...중략

    return !( ((d1<0)||(d2<0)||(d3<0)) && ((d1>0)||(d2>0)||(d3>0)) );
}
```

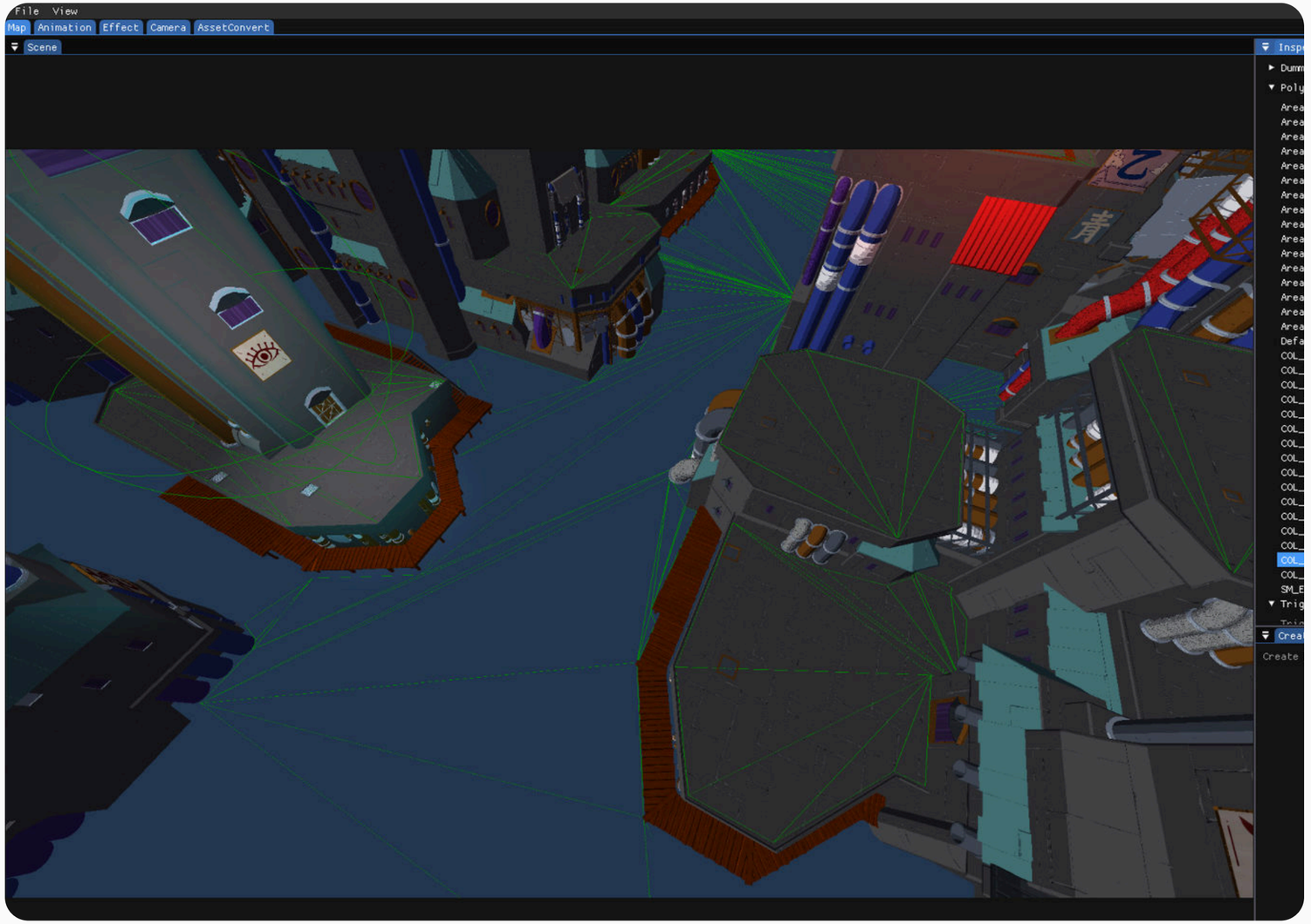
*삼각형 내부에 다른 정점이 있으면 Ear가 아님을 판별하는 함수
외적을 통해 세 변의 부호가 모두 같으면 내부로 판별함

02. System 다각형 Navigation Cell

활용 - Polygon With Hole



▲ 툴에서의 예시 결과물 스크린샷



▲ 실제 프로젝트에서 생성된 Navigation Cell

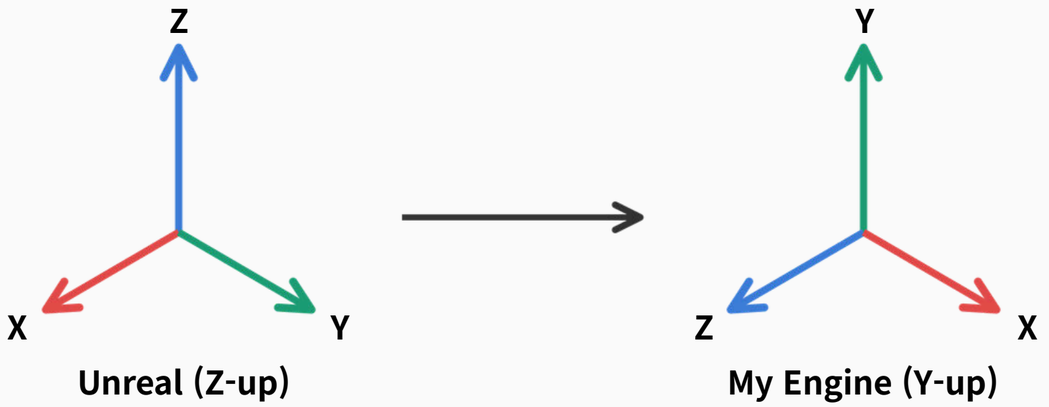
02. System 언리얼 맵 데이터 좌표계 변환

왜?

- 외부 툴에서 뽑은 맵배치 데이터와 자체엔진의 좌표계 표현이 달라 **위치·회전·스케일이 뒤틀리는 문제** 발생
- 축 스왑뿐만 아니라 **회전(쿼터니언)까지** 올바르게 변환해야함

효과

- 외부 툴 데이터에서 만든 맵 데이터를 **자체 엔진에 바로 활용**
- 직접 배치하는 **반복적인 작업을 줄여 작업 효율 상승**




```
void CUEMapdataParser::Change_SRT(OUT SRT_DATA& tSRT_Data)
{
    Matrix ScaleMatrix    = Matrix::CreateScale(tSRT_Data.vScale);
    Matrix TransMatrix    = Matrix::CreateTranslation(tSRT_Data.vPosition * m_fMulScale);
    Matrix RotationMatrix = Matrix::CreateFromQuaternion(tSRT_Data.vQuat);

    Matrix InvB = m_ChangeMatrix.Transpose();

    Matrix WorldMatrix = ScaleMatrix * RotationMatrix * TransMatrix;

    WorldMatrix = InvB * WorldMatrix * m_ChangeMatrix;

    WorldMatrix.Decompose(tSRT_Data.vScale, tSRT_Data.vQuat, tSRT_Data.vPosition);
}
```

```
const Matrix  m_ChangeMatrix  =
{
   1,  0,  0,  0,
   0,  0, -1,  0,
   0,  1,  0,  0,
   0,  0,  0,  1
};
```

*변환 행렬 B
Y와 Z를 교환하여 부호 처리

변환 행렬의 역 = 전치

기저 변환으로 좌표계 변환
$B^{-1} \cdot World \cdot B$

변환된 행렬에서 S/R/T 다시 분해

02. System 언리얼 맵 데이터 좌표계 변환



02. System 애니메이션 블렌딩

왜?

- 애니메이션을 A에서 B로 즉시 바꾸면 자세가 튀어 동작이 딱딱하게 끊김
- 전환 순간 두 동작을 자연스럽게 이어 붙일 방법이 필요했음

효과

- 정지 자세 사이를 단순 보간하는 방식과 달리, 전환 구간에서 두 애니메이션을 도시에 재생하며 각 시점의 포즈를 본별로 섞어 자연스럽게 전환
- 두 동작이 살아있는 채로 겹쳐 이어지므로, 전환 중에도 움직이 끊기지 않음



02. System 애니메이션 블렌딩

```
void CModel::Blend_Animation(_float fTimeDelta, _float fRatio)
{
    m_vecAnimations[m_iPrevAnimIndex]->SetUp_PoseDatasForBlending(m_vecPrevAnimationPose, fTimeDelta);
    m_vecAnimations[m_iCurrentAnimIndex]->SetUp_PoseDatasForBlending(m_vecCurrAnimationPose, fTimeDelta);

    _uint i = 0;
    for (auto& pBone : m_vecBones)
    {
        _vector vScale, vQuaternion, vTranslation;

        vScale = XMVectorLerp(
            XMLoadFloat3(&m_vecPrevAnimationPose[i].vScale),
            XMLoadFloat3(&m_vecCurrAnimationPose[i].vScale), fRatio);

        vQuaternion = XMQuaternionSlerp(
            XMLoadFloat4(&m_vecPrevAnimationPose[i].vQuaterion),
            XMLoadFloat4(&m_vecCurrAnimationPose[i].vQuaterion), fRatio);

        vTranslation = XMVectorLerp(
            XMLoadFloat3(&m_vecPrevAnimationPose[i].vTranslation),
            XMLoadFloat3(&m_vecCurrAnimationPose[i].vTranslation), fRatio);

        _matrix matTransformation = XMMatrixAffineTransformation(
            vScale, XMVectorSet(0,0,0,1), vQuaternion, vTranslation);
        pBone->Set_TransformationMatrix(matTransformation);
        ++i;
    }

    for (size_t i = 0; i < m_vecBones.size(); ++i)
        m_vecBones[i]->Update_CombinedTransformMatrix(m_vecBones, XMLoadFloat4x4(&m_matPreTransform));
}
```

이전·현재 애니메이션의 포즈를 각각 추출

Scale 선형 보간

Rotation 구면 보간

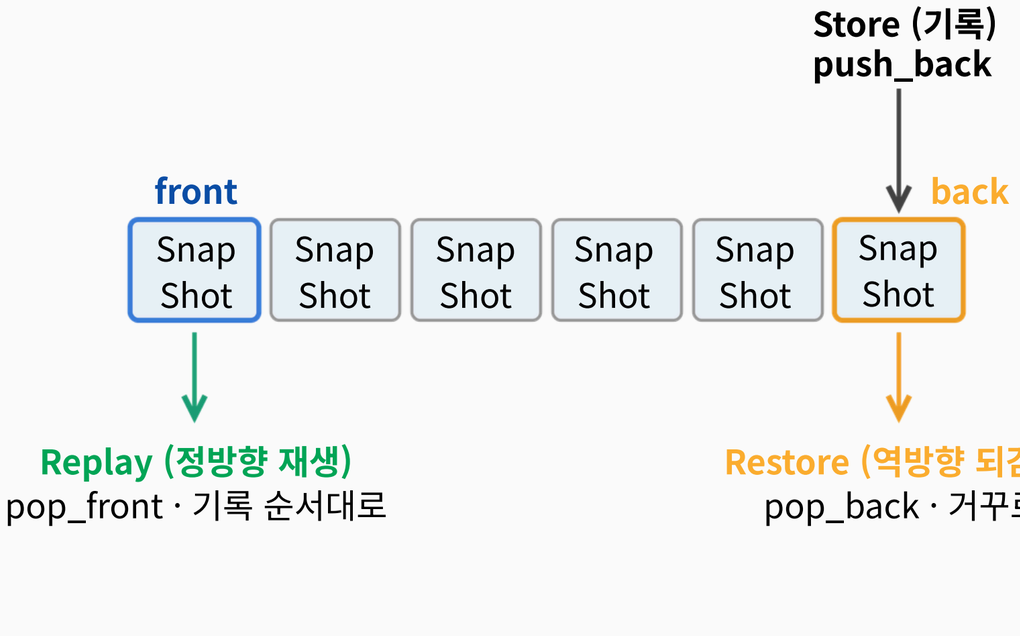
Translation 선형 보간

보간한 S/R/T로 본 행렬 재구성

본 계층 최종 행렬 갱신

03. Contents 리플레이







어떻게

- **Memento 패턴을 활용**해 각 오브젝트가 자신의 상태를 Memento로 캡슐화해 저장 / 복원
- 기록된 스냅샷은 Manager가 **하나의 Deque**으로 관리

왜 Deque?

- FIFO를 하게되면 다시보기, LIFO를 하게되면 시간 되돌리기, **두가지 기능을 하나의 자료구조로 처리가 가능**하기 때문
- 양쪽 끝 삽입·삭제가 모두 **O(1)으로 양방향 접근에 적합**

03. Contents 리플레이

```
unordered_map<ObjectType, deque<CMemento>> m_Snapshots;

void CMementoManager::Store()
{
    // ...중략
    m_Snapshots[ObjectType::PLAYER].push_back(pPlayer->CreateMemento());
}

void CMementoManager::Replay()
{
    // ...중략
    pPlayer->Restore(m_Snapshots[ObjectType::PLAYER].front());
    m_Snapshots[ObjectType::PLAYER].pop_front();
}

void CMementoManager::Restore()
{
    // ...중략
    pPlayer->Restore(m_Snapshots[ObjectType::PLAYER].back());
    m_Snapshots[ObjectType::PLAYER].pop_back();
}
```

*오브젝트 타입별로 순회하며 기록된 스냅샷을 저장·다시보기·되돌리기하는 함수

```
CMemento CObject::CreateMemento()
{
    return CMemento(GetPos(), GetDir(),
                    pBitmap->GetAnimKey(),
                    pAnimComp->GetCurrentFrameIndex());
}
```

*오브젝트가 자신의 상태를 **Memento로 캡슐화해 생성하는 함수**
위치·방향·애니메이션 키·프레임 인덱스를 스냅샷으로 저장

```
void CObject::Restore(const CMemento& _m)
{
    pAnimComp->PlayAnimation(_m.AnimKey, false);
    pAnimComp->SetCurrentFrameIndex(_m.FrameIndex);
    SetPos(_m.Position);
    SetDir(_m.Direction);
}
```

*메멘토로부터 **상태 복원하는 함수**

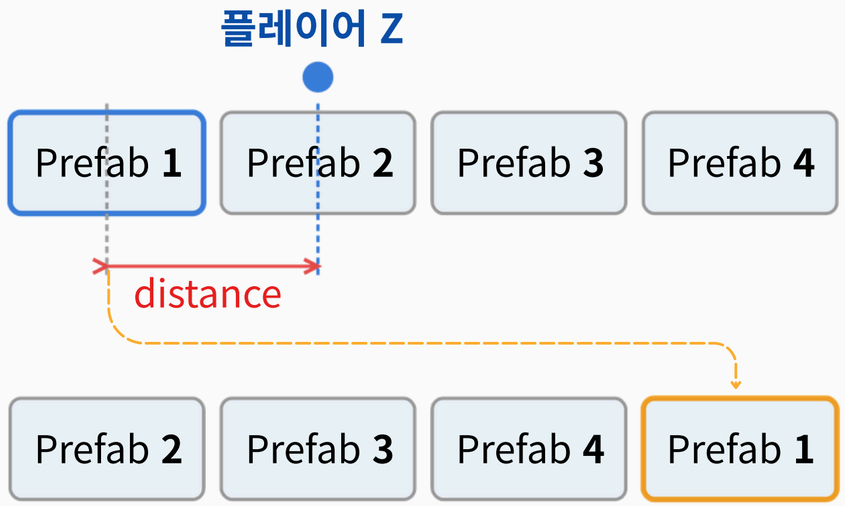
```
void CScene::Store()
{
    m_fElapsedTime += CTimeManager::Get_Instance()->GetDeltaTime();
    if (m_fElapsedTime >= CAPTURETIME)
    {
        CMementoManager::Get_Instance()->Store();
        m_fElapsedTime -= CAPTURETIME;
    }
}
```

*CAPTURETIME, **매 틱 저장대신 델타타임을 누적해 저장**하여 메모리 과다사용을 방지

03. Contents 무한맵



*Prefab을 Z방향으로 일정히 나열



무엇을

- 도로·건물을 하나의 **Prefab 단위로 묶어, 고정 개수의 Prefab을 순환 재활용해 무한 맵 구현**

어떻게?

- 플레이어가 맨 앞 Prefab을 일정 거리 지나가면, 그 Prefab을 앞에서 떼어(pop_front) 맨 뒤 위치로 옮겨 재연결(push_back)
- **노드를 떼어 붙이는 것만으로 순서가 유지되도록 list**로 관리

03. Contents 무한맵

```
void CPrefabRecycleSystem::Update()
{
    const _float fStandard_Z = m_pPrefabList.front()->GetTransform()->Get_Info(INFO_POS).z;
    const _float fPlayerPosition_Z = m_pPlayer->Get_Position().z;

    if (fPlayerPosition_Z - fStandard_Z >= m_fDistance)
        Sorting();
}
```

플레이어가 Prefab을 m_fDistance만큼 지나가면 재활용

```
void CPrefabRecycleSystem::Sorting()
{
    CGameObject* pWasFront = m_pPrefabList.front();
    CGameObject* pBack     = m_pPrefabList.back();

    m_pPrefabList.pop_front();

    _vec3 vNewPosition = pBack->GetTransform()->Get_Info(INFO_POS);
    vNewPosition.z += m_fDistance;
    pWasFront->GetTransform()->Set_Info(INFO_POS, vNewPosition);
    static_cast<CPrefab*>(pWasFront)->Set_ChildrensMatrix();

    m_pPrefabList.push_back(pWasFront);

    RandomSpawnMonster(vNewPosition);
}
```

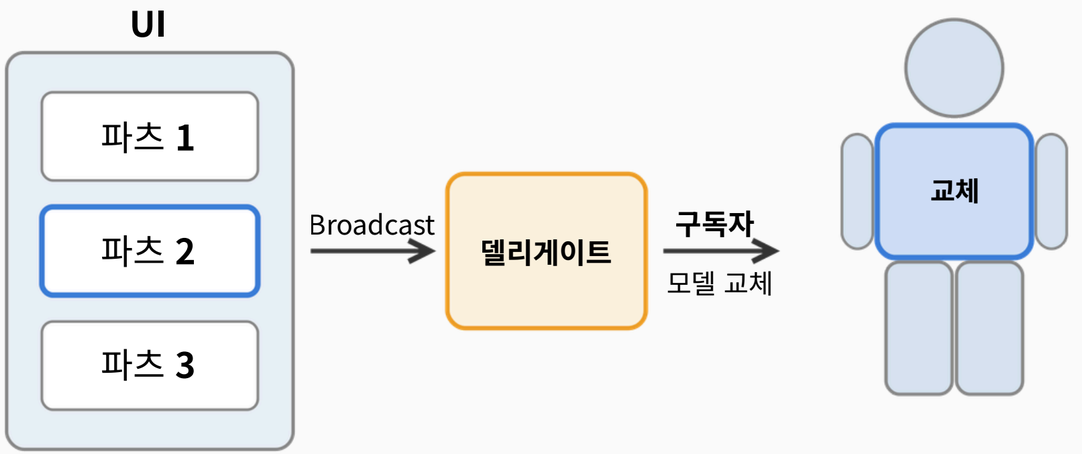
앞쪽 Prefab을 List에서 떼어냄

List 맨 뒤 Prefab의 다음 위치로 이동

List 맨 뒤로 재연결 하여 재활용

03. Contents 캐릭터 커스텀





어떻게

- **하나의 스켈레톤에 여러 파츠 메시를 스키닝** → 파츠를 교체해도 같은 본을 공유하므로 애니메이션이 그대로 적용
- 파츠 교체는 UI 선택 → 델리게이트로 신호 → 해당 부위 메시만 교체

03. Contents 캐릭터 커스텀

```
m_pMasterMesh = static_cast<CMesh>(pMaster->m_vecMeshes[0]->Clone(nullptr));

Add_Parts(pPrototypeTags->wstrHead);
Add_Parts(pPrototypeTags->wstrFace);
//... 중략
```

마스터 모델의 본·메시를 클론
→ 공유 스켈레톤 확보

커스텀 레벨에서 확정된 파츠들을 플레이어의
Model Initialize 시점에 전달 받음

```
HRESULT CModel::Add_Parts(const wstring& wstrPrototypeTag)
{
    CModel* pPart = Get_Clone(wstrPrototypeTag);

    for (auto& pMesh : pPart->m_vecMeshes)
    {
        CMesh* pClone = static_cast<CMesh>(pMesh->Clone(nullptr));
        m_vecMeshes.push_back(pClone);
    }
    // ...중략
}
```

각 파츠에서 메시만 가져와 하나의 모델에 누적

```
HRESULT CCustomObject::Render()
{
    pModel->Bind_Bones(pShader);

    m_vecPartObjects[HEAD]->Render();
    m_vecPartObjects[FACE]->Render();

    if (m_bSelectedOneset)
        m_vecPartObjects[ONESET]->Render();
    else {
        m_vecPartObjects[UPPER]->Render();
        m_vecPartObjects[LOWER]->Render();
    }
}
```

객체를 그리는 시점에 모델의 마스터본을 셰이더에 바인딩

각 파츠 메시를 같은 마스터 본으로 렌더

03. Contents 벽타기



무엇을

- 조준한 벽 지점으로 와이어를 발사하여 벽에 붙어 이동하는 상태

어떻게?

- 3단계 상태로 구분하여 구현

→ 감지 : 카메라 방향으로 레이캐스트를 쏴 와이어를 걸 벽 충돌 지점을 찾음
→ 발사 : 지점을 향해 날아가며 벽에 도착했는지 판정
→ 이동 : 벽 법선을 기준으로 캐릭터 Up을 정렬해 벽면을 이동

03. Contents 벽타기

```
_bool CState_WireJumpReady::Check_Wire()
{
    // ...중략
    _vector vCamPos  = pCameraTransform->Get_Info(POS);
    _vector vCamLook = pCameraTransform->Get_Info(LOOK);

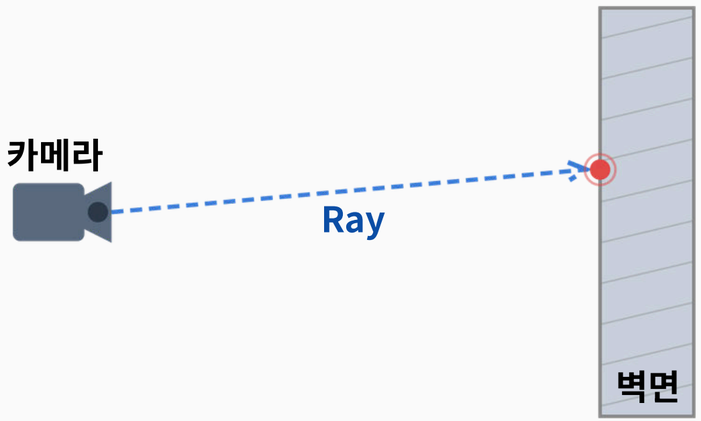
    m_pRopeRay->Setup_Ray(vCamPos, XMVector3Normalize(vCamLook));

    COLMESH_HITINFO wallHit = {};
    const _float fMaxRopeDistance = 30.f;
    if (pColMeshManager->Raycast_World(m_pRopeRay, fMaxRopeDistance, wallHit) == false)
        return false;

    ropeInfo.vHitPosition = wallHit.vHitPosition;
    // ...중략
    return true;
}
```

현재 카메라의 위치·방향 가져오기

카메라 Look 방향으로 레이 세팅 후 발사
최대거리 fMaxRopeDistance 안에서 벽 충돌 검사



03. Contents 벽타기

```
_bool CState_Launch::Is_LandToWall()
{
    // ...중략
    _vector vHitPosition = XMLoadFloat3(&m_pCurrentInfo->vHitPosition);
    _vector vNormal      = XMLoadFloat3(&wallSpaceInfo.vTargetUp);

    _vector vDiff = vPlayerPos - vHitPosition;

    _float fDist = XMVectorGetX(XMVector3Dot(vDiff, vNormal));

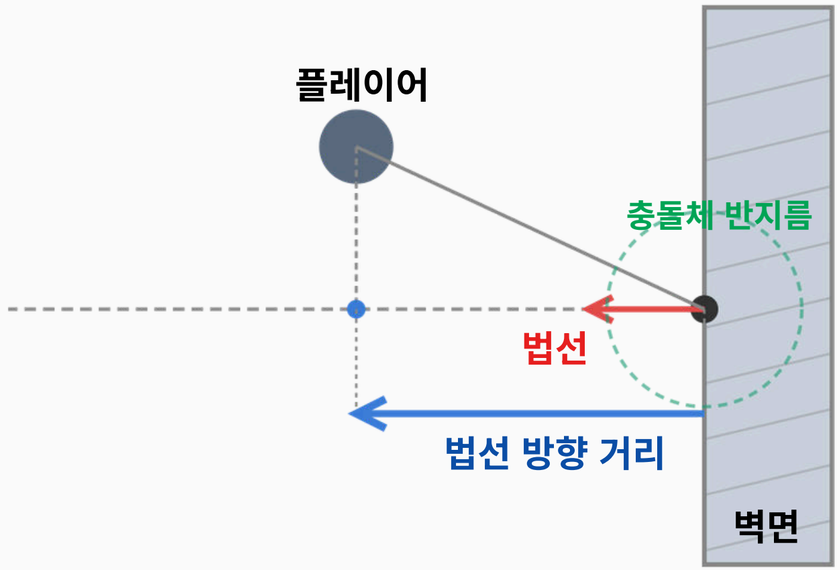
    if (fDist <= fRadius)
        return true;

    return false;
}
```

(플레이어 위치) - (벽과 Ray 충돌 지점)

벽 법선에 투영한 거리 = 벽면 수직 거리

반지름이하로 가까워지면 도착 판정



03. Contents 벽타기

```
void CMainPlayer::Movement_Wall(const _float fTimeDelta)
{
    // ...중략
    _float3 vScale = pTransform->Get_Scaled();

    _vector vPrevRight = ::XMVector3Normalize(pTransform->Get_Info(TRANSFORM_INFO_STATE::RIGHT));
    _vector vPrevUp = ::XMVector3Normalize(pTransform->Get_Info(TRANSFORM_INFO_STATE::UP));
    _vector vPrevLook = ::XMVector3Normalize(pTransform->Get_Info(TRANSFORM_INFO_STATE::LOOK));

    _vector vCellUp = pNavigation->Get_CellNormal_World();
    _vector vPrevPosition = pTransform->Get_Info(TRANSFORM_INFO_STATE::POS);

    _vector vNewLook = vPrevLook - ::XMVector3Dot(vPrevLook, vCellUp) * vCellUp;

    if (::XMVectorGetX(::XMVector3LengthSq(vNewLook)) <= g_XMEpsilon.f[0])
    {
        vNewLook = vPrevRight - ::XMVector3Dot(vPrevRight, vCellUp) * vCellUp;
    }

    vNewLook = ::XMVector3Normalize(vNewLook);
    _vector vNewRight = ::XMVector3Normalize(::XMVector3Cross(vCellUp, vNewLook));

    pTransform->Set_Info(TRANSFORM_INFO_STATE::RIGHT, vNewRight * vScale.x);
    pTransform->Set_Info(TRANSFORM_INFO_STATE::UP, vCellUp * vScale.y);
    pTransform->Set_Info(TRANSFORM_INFO_STATE::LOOK, vNewLook * vScale.z);

    // ...중략
}
```

현재 캐릭터 축

벽 셀의 법선

Look에서 법선 성분 제거 → 벽면에 평행한 Look (투영)

Look이 법선과 평행하다면 Right로 대체

외적으로 Right 완성 (직교 좌표계)
플레이어 축을 벽 기준으로 재설정 (Up = 법선)

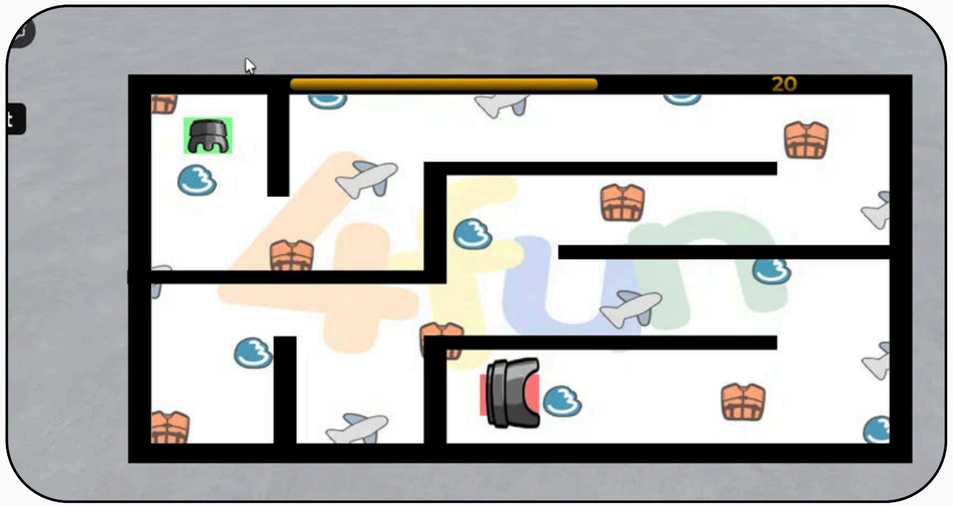


04. 그 외 4Fun Speaking - 로블록스 기반 언어 학습 게임



▲ 개발했던 게임들의 썸네일 아이콘

로블록스 스튜디오 기반으로 소수의 인원과 협업하여 외국어 회화를 위한 미니게임을 개발했었습니다.
게임 개발자의 꿈을 놓고 싶지 않아 퇴근 후 개발한 작은 볼륨의 게임들입니다. 클리커 게임과 오브젝트와의 간단한 상호작용 위주의 게임들이며 **실제 학생들에게 테스트 서비스를 하며 피드백을 받고 의견을 주고 받은 경험**이 신선했던 프로젝트입니다.





강병준

가져다 쓰는 개발자가 아니라,
만들어 이해하는 개발자가 되겠습니다.

CONTACT

010-2653-1292

dzzzing@naver.com